SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Shaik Basheer	**Reg. No.:**	192465049
Section / Batch:	CSE – AI/2024	**Date:**	22-08-26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Q33. Kruskal vs Boruvka Algorithm (MST Comparison)

Given a graph, construct MST using Kruskal and Boruvka algorithms. Compare edge selection process. Analyse efficiency for different graph types. Evaluate computational complexity. Discuss scalability. Generate insights on MST algorithms.

Instructions: Solve the assigned problem, show the algorithm/steps clearly, analyze correctness, time and space complexity, and provide the requested comparison or insights.

1. Introduction

A Minimum Spanning Tree (MST) of a connected, weighted, undirected graph is a spanning tree that connects all vertices with the minimum possible total edge weight.

Kruskal's and Boruvka's algorithms are greedy algorithms used to find an MST. Kruskal sorts all edges globally and adds the smallest edge when it does not form a cycle. Boruvka works with multiple components and repeatedly selects the cheapest outgoing edge from every component.

This assessment compares their edge selection process, intermediate results, correctness, time and space complexity, efficiency for different graph types, scalability, and practical suitability.

2. Given Weighted Graph

Vertices: V = {A, B, C, D, E, F}

Weighted Edges:

A–B = 4

A–C = 2

B–C = 1

B–D = 5

C–D = 8

C–E = 10

D–E = 2

D–F = 6

E–F = 3

B–E = 7

C–F = 9

The graph contains 6 vertices and 11 edges.

3. Kruskal's Algorithm

Kruskal's algorithm sorts all edges in non-decreasing order of weight and considers them one by one. An edge is selected if it does not create a cycle. A Disjoint Set Union (DSU), or Union-Find, is used for efficient cycle detection.

Steps:

1. Sort all edges by increasing weight.
2. Create a separate component for every vertex.
3. Select the smallest remaining edge.
4. Check whether its endpoints belong to different components.
5. If they are different, add the edge and merge the components.
6. If they are already connected, reject the edge.
7. Repeat until V − 1 edges are selected.

4. Kruskal Intermediate Results

Sorted order:

B–C = 1, A–C = 2, D–E = 2, E–F = 3, A–B = 4, B–D = 5, D–F = 6, B–E = 7, C–D = 8, C–F = 9, C–E = 10.

Step 1: B–C = 1 → Selected → Weight = 1

Step 2: A–C = 2 → Selected → Weight = 3

Step 3: D–E = 2 → Selected → Weight = 5

Step 4: E–F = 3 → Selected → Weight = 8

Step 5: A–B = 4 → Rejected because it forms cycle A–C–B

Step 6: B–D = 5 → Selected → Weight = 13

Kruskal MST = {B–C, A–C, D–E, E–F, B–D}

Total MST Weight = 13

5. Boruvka's Algorithm

Boruvka's algorithm starts with every vertex as a separate component. In each phase, every component finds its cheapest outgoing edge. The selected safe edges are added and the connected components are merged.

- Steps:
  1. Initially, every vertex is a separate component.
  2. For every component, find its cheapest outgoing edge.
  3. Select the cheapest outgoing edges.
  4. Add the selected edges to the MST.
  5. Merge the connected components.
  6. Repeat until only one component remains.

6. Boruvka Intermediate Results

Initial components:

{A}, {B}, {C}, {D}, {E}, {F}

Phase 1:

{A} selects A–C = 2

{B} selects B–C = 1

{C} selects B–C = 1

{D} selects D–E = 2

{E} selects D–E = 2

{F} selects E–F = 3

Unique selected edges:

A–C = 2, B–C = 1, D–E = 2, E–F = 3

New components:

{A, B, C} and {D, E, F}

Current weight = 8

Phase 2:

The cheapest outgoing edge between the two components is B–D = 5.

Select B–D = 5.

All vertices are connected.

Boruvka MST = {A–C, B–C, D–E, E–F, B–D}

Total MST Weight = 13

7. Edge Selection Comparison

Kruskal follows a global edge-based strategy:

B–C → A–C → D–E → E–F → A–B (rejected) → B–D

Boruvka follows a component-based strategy:

Phase 1 → A–C, B–C, D–E, E–F

Phase 2 → B–D

Kruskal = globally sorted edge selection.

Boruvka = cheapest outgoing edge from every component.

8. Correctness Analysis

Kruskal is correct because it selects safe minimum-weight edges connecting different components. The cut property guarantees that such an edge can belong to an MST. Union-Find prevents cycles.

Boruvka is correct because the cheapest outgoing edge of each component is a safe edge according to the cut property. Repeated component merging eventually connects all vertices without cycles, producing an MST.

9. Time and Space Complexity

Kruskal:

Sorting = O(E log E)

Union-Find operations ≈ O(E α(V))

Overall = O(E log E)

Space = O(V + E)

Boruvka:

The number of components decreases substantially in each phase, giving O(log V) phases. With efficient edge processing, the overall complexity is commonly O(E log V).

Space = O(V + E).

10. Efficiency for Different Graph Types

Sparse Graph:

Kruskal performs well because relatively few edges need to be sorted. Boruvka is also effective because components merge quickly.

Dense Graph:

Kruskal may spend considerable time sorting a very large edge set. Boruvka can be attractive, particularly when component processing is parallelized.

Very Large Graph:

Boruvka has strong scalability potential because components can independently identify their cheapest outgoing edges. Kruskal remains simple but depends on global edge sorting.

11. Scalability Analysis

Kruskal:

• Requires global sorting.

• Sorting becomes expensive as the number of edges increases.

• Union-Find gives efficient cycle detection.

• Straightforward but has a global dependency on the sorted edge list.

Boruvka:

• Processes multiple components.

• Each component independently searches for its cheapest outgoing edge.

• Components reduce rapidly across phases.

• Well suited to parallel and distributed graph processing.

12. Executable Python Code

main.py

```python
edges = [
    ('A','B',4), ('A','C',2), ('B','C',1), ('B','D',5),
    ('C','D',8), ('C','E',10), ('D','E',2), ('D','F',6),
    ('E','F',3), ('B','E',7), ('C','F',9)
]
vertices = ['A','B','C','D','E','F']
def find(parent, x):
    if parent[x] != x:
        parent[x] = find(parent, parent[x])
    return parent[x]
def union(parent, rank, a, b):
    ra, rb = find(parent,a), find(parent,b)
    if ra == rb:
        return False
    if rank[ra] < rank[rb]:
        parent[ra] = rb
    elif rank[ra] > rank[rb]:
        parent[rb] = ra
    else:
        parent[rb] = ra
        rank[ra] += 1
    return True
```

input

```
Kruskal Total Weight: 13
Boruvka MST: [('A', 'C', 2), ('B', 'C', 1), ('D', 'E', 2), ('E', 'F', 3), ('B', 'D', 5)]
Boruvka Total Weight: 13


...Program finished with exit code 0
Press ENTER to exit console.
```

Both algorithms produce an MST with the same minimum total weight for the given graph.

13. Overall Comparison

Parameter | Kruskal | Boruvka

Approach | Edge-oriented | Component-oriented

Selection | Global minimum safe edge | Cheapest outgoing edge per component

Starting vertex | Not required | Not required

Cycle detection | Union-Find | Union-Find

Parallelism | Limited | High potential

Sparse graphs | Very effective | Effective

Dense graphs | Sorting can be costly | Can be attractive

Scalability | Good | Very good with parallelism

Time | O(E log E) | O(E log V)

Space | O(V + E) | O(V + E)

Given MST weight | 13 | 13

14. Advantages and Limitations

Kruskal Advantages:

1. Simple and easy to understand.
2. Effective for sparse graphs.
3. Does not require a starting vertex.
4. Union-Find provides efficient cycle detection.

Kruskal Limitations:

1. Requires sorting all edges.
2. Sorting can be expensive for dense graphs.
3. Large edge lists require storage.

Boruvka Advantages:

1. Does not require a starting vertex.
2. Component-based selection supports parallelism.
3. Components merge rapidly.
4. Suitable for large-scale graph processing.

Boruvka Limitations:

1. More complex to implement.
2. Requires repeated identification of cheapest outgoing edges.
3. Efficient implementation is important for good performance.

15. Critical Evaluation

Kruskal is preferable when the graph is naturally represented as an edge list and the number of edges is manageable. Its sorting and Union-Find operations make it reliable and straightforward.

Boruvka provides a scalability advantage because multiple components can independently identify their cheapest outgoing edges. This makes it attractive for parallel and distributed environments.

Therefore, algorithm choice should depend on graph size, density, representation, available processing resources, and whether parallel execution is required.

16. Result Interpretation

For the given graph, both algorithms produce:

MST = {A–C, B–C, D–E, E–F, B–D}

Total MST Weight = 2 + 1 + 2 + 3 + 5 = 13

Kruskal reaches the result through globally sorted edges, while Boruvka reaches it through component-wise cheapest outgoing edges. The equal MST weight validates that both greedy strategies correctly solve the MST problem for this graph.

17. Synthesis and Insights

1. Both Kruskal and Boruvka are greedy MST algorithms.
2. Both produce the same minimum total weight for the given graph.
3. Kruskal is edge-oriented and depends on global sorting.
4. Boruvka is component-oriented and repeatedly chooses the cheapest outgoing edge.
5. Kruskal is particularly effective for sparse graphs with manageable edge lists.
6. Boruvka has strong potential for parallel and distributed processing.
7. Graph structure and implementation environment should determine the algorithm choice.
8. Neither algorithm is universally superior.

18. Conclusion

Kruskal and Boruvka algorithms solve the Minimum Spanning Tree problem using different greedy strategies. Kruskal sorts all edges and selects the cheapest safe edges, whereas Boruvka repeatedly selects the cheapest outgoing edge from every component.

For the given graph, both algorithms produce the MST {A–C, B–C, D–E, E–F, B–D} with total weight 13.

Kruskal is simple and effective for sparse graphs. Boruvka is attractive for component-based, parallel, and distributed processing. The final choice should depend on graph density, graph representation, graph size, and available computational resources.

Final Insight:

Kruskal = select the cheapest safe edges globally.

Boruvka = let every component select its cheapest outgoing edge.

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department

Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____